

Question 1: What is Anomaly Detection? Explain its types (point, contextual, and collective anomalies) with examples.

Ans:- **Anomaly Detection** is the process of identifying **unusual or abnormal data points** that significantly differ from the majority of the data. These unusual observations are called **anomalies** or **outliers** and may indicate fraud, system failures, network intrusions, or data errors.

Types of Anomalies

1. Point Anomaly

A **point anomaly** is a single data point that is significantly different from the rest of the dataset.

Example:

- In a bank transaction dataset, most transactions are between **₹500 and ₹20,000**, but one transaction is **₹10,00,000**. This transaction is a point anomaly and may indicate fraud.
-

2. Contextual Anomaly

A **contextual anomaly** is a data point that is abnormal only in a specific context, such as time, location, or season.

Example:

- A temperature of **35°C** is normal in summer but unusual during winter in the same city.
 - Similarly, high website traffic at **2:00 AM** may be abnormal, even if the same traffic is normal during the afternoon.
-

3. Collective Anomaly

A **collective anomaly** occurs when a group of related data points is abnormal, even though individual data points may appear normal.

Example:

- A sequence of multiple failed login attempts from the same IP address within a short time may indicate a cyberattack.
- Each failed login alone is normal, but the entire sequence is considered anomalous.

Question 2: Compare Isolation Forest, DBSCAN, and Local Outlier Factor in terms of their approach and suitable use cases.

Ans:-

Algorithm	Approach	Advantages	Suitable Use Cases
Isolation Forest	Randomly isolates data points using decision trees. Anomalies are isolated in fewer splits than normal points.	Fast, efficient for large and high-dimensional datasets, does not rely on distance calculations.	Fraud detection, network intrusion detection, financial transactions, large-scale anomaly detection.
DBSCAN	Density-based algorithm that groups dense regions and labels low-density points as outliers (noise).	Detects arbitrarily shaped clusters, identifies noise automatically, no need to specify the number of clusters.	Geographic data analysis, image processing, customer segmentation, detecting spatial outliers.

Local Outlier Factor (LOF)	Compares the local density of a data point with the densities of its neighbors. Points with much lower density are considered anomalies.	Detects local anomalies effectively, works well when data density varies across regions.	Credit card fraud detection, healthcare anomaly detection, sensor monitoring, network security.
-----------------------------------	--	--	---

Question 3: What are the key components of a Time Series? Explain each with one example.

Ans:- A **Time Series** is a sequence of observations recorded at regular intervals over time (e.g., daily, monthly, yearly). Time series data usually consists of four main components.

1. Trend

The **trend** is the long-term movement of data, showing whether the values generally increase, decrease, or remain constant over time.

Example:

- An e-commerce company's annual sales steadily increase over the last five years due to business growth.
-

2. Seasonality

Seasonality refers to regular and repeating patterns that occur at fixed time intervals, such as daily, monthly, quarterly, or yearly.

Example:

- Ice cream sales increase every summer and decrease during winter.
 - Online shopping sales rise significantly during festivals like Diwali or Christmas each year.
-

3. Cyclical Component

The **cyclical component** represents long-term fluctuations caused by economic or business cycles. Unlike seasonality, these cycles do not occur at fixed intervals.

Example:

- Automobile sales increase during periods of economic growth and decline during economic recessions.
-

4. Irregular (Random) Component

The **irregular** or **random component** consists of unpredictable variations caused by unexpected events that cannot be explained by trend, seasonality, or cycles.

Example:

- A sudden drop in airline bookings due to a natural disaster or pandemic.
- Unexpected spikes in online orders during a flash sale.

Question 4: Define Stationary in time series. How can you test and transform a non-stationary series into a stationary one?

Ans:- A **stationary time series** is a time series whose **statistical properties remain constant over time**. This means that the **mean, variance, and autocovariance** do not change with time.

Many forecasting models, such as **ARIMA**, require the data to be stationary for accurate predictions.

Characteristics of a Stationary Time Series

- **Constant Mean:** The average value remains the same over time.
- **Constant Variance:** The spread of the data does not change over time.
- **Constant Autocorrelation:** The relationship between observations depends only on the time lag, not on the actual time.

Example

- **Stationary:** Daily temperature fluctuations around a stable average.
 - **Non-Stationary:** Company sales that continuously increase over the years due to business growth.
-

How to Test for Stationarity

1. Visual Inspection

Plot the time series and observe:

- Whether the mean changes over time.
- Whether the variance increases or decreases.
- Whether there is a visible trend or seasonality.

2. Augmented Dickey-Fuller (ADF) Test

The **ADF test** is the most commonly used statistical test for stationarity.

Hypotheses:

- **Null Hypothesis (H_0):** The series is **non-stationary**.
- **Alternative Hypothesis (H_1):** The series is **stationary**.

Decision Rule:

- **p-value < 0.05:** Reject $H_0 \rightarrow$ The series is **stationary**.
- **p-value $\geq$ 0.05:** Fail to reject $H_0 \rightarrow$ The series is **non-stationary**.

3. KPSS Test

Another statistical test where:

- **Null Hypothesis (H_0):** The series is **stationary**.
- **Alternative Hypothesis (H_1):** The series is **non-stationary**.

Using both ADF and KPSS together provides a more reliable assessment.

How to Transform a Non-Stationary Series into a Stationary One

1. Differencing

Subtract each observation from the previous one to remove the trend.

$$Y'_t = Y_t - Y_{t-1} \quad Y'_t = Y_t - Y_{t-1}$$

This is the most common method for making a series stationary.

2. Log Transformation

Apply a logarithm to stabilize variance when the data shows exponential growth.

$$Y' = \log(Y)$$

3. Seasonal Differencing

Remove seasonal patterns by subtracting observations from the previous season.

$$Y'_t = Y_t - Y_{t-s}$$

where **s** is the seasonal period (e.g., 12 for monthly data).

4. Detrending

Remove the long-term trend using regression or moving averages.

Question 5: Differentiate between AR, MA, ARIMA, SARIMA, and SARIMAX models in terms of structure and application.

Ans:- Time series forecasting models differ based on how they capture **past values, past errors, trends, seasonality, and external variables**.

Model	Structure	Application
AR (AutoRegressive)	Uses past values of the time series to predict future values. Represented as AR(p) ,	Suitable for stationary time series where current values depend on previous values (e.g.,

where **p** is the number of lag observations.

stock prices, temperature).

MA (Moving Average)

Uses past forecast errors (residuals) to predict future values. Represented as **MA(q)**, where **q** is the number of previous error terms.

Suitable when prediction errors influence future observations, such as short-term demand forecasting.

ARIMA (AutoRegressive Integrated Moving Average)

Combines **AR(p)**, **Differencing (d)**, and **MA(q)**. Represented as **ARIMA(p, d, q)**.

Used for **non-seasonal, non-stationary** time series after converting them to stationary through differencing.

SARIMA (Seasonal ARIMA)

Extends ARIMA by adding seasonal components. Represented as **SARIMA(p, d, q)(P, D, Q, s)**.

Suitable for **seasonal time series**, such as monthly sales, electricity demand, or tourism data.

SARIMAX (Seasonal ARIMA with

Extends SARIMA by including external (exogenous) variables

Used when external factors (e.g., promotions, holidays, weather, economic

Exogenous Variables) that influence the target series. indicators) affect forecasts.

Summary of Each Model

1. AR (AutoRegressive)

- Predicts future values using previous observations.
- Works only with **stationary** data.

Example: Predicting today's stock price using prices from the previous few days.

2. MA (Moving Average)

- Predicts future values using previous forecast errors.
- Suitable for modeling random fluctuations.

Example: Forecasting sales while accounting for previous prediction errors.

3. ARIMA

- Combines **AR + Differencing + MA**.
- Handles **non-stationary** data by first making it stationary.

Example: Forecasting yearly company revenue with a trend but no seasonality.

4. SARIMA

- Adds seasonal components to ARIMA.
- Captures repeating seasonal patterns.

Example: Forecasting monthly retail sales that increase every festive season.

5. SARIMAX

- Includes seasonal patterns **and** external variables.
- Produces more accurate forecasts when outside factors influence the data.

Example: Forecasting product sales using historical sales, holiday information, and advertising expenditure.

Dataset:

- NYC Taxi Fare Data
- AirPassengers Dataset Question 6: Load a time series dataset (e.g., AirPassengers), plot the original series, and decompose it into trend, seasonality, and residual components (Include your Python code and output in the code box below.)

Ans:- # Import required libraries

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from statsmodels.datasets import get_rdataset
```

```
from statsmodels.tsa.seasonal import seasonal_decompose
```

```
# Load the AirPassengers dataset
data = get_rdataset("AirPassengers").data

# Create a datetime index
data["time"] = pd.date_range(start="1949-01", periods=len(data),
                              freq="MS")
data.set_index("time", inplace=True)

# Select the passenger series
ts = data["value"]

# Plot the original time series
plt.figure(figsize=(10, 4))
plt.plot(ts)
plt.title("AirPassengers Time Series")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.grid(True)
plt.show()

# Decompose the time series
```

```
decomposition = seasonal_decompose(ts, model="multiplicative",  
period=12)
```

```
# Plot the decomposed components
```

```
decomposition.plot()
```

```
plt.show()
```

Question 7: Apply Isolation Forest on a numerical dataset (e.g., NYC Taxi Fare) to detect anomalies. Visualize the anomalies on a 2D scatter plot. (Include your Python code and output in the code box below.)

Ans:- # Import required libraries

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.ensemble import IsolationForest
```

```
# Load the NYC Taxi Fare dataset
```

```
# Replace the filename with the correct path
```

```
data = pd.read_csv("nyc_taxi.csv")
```

```
# Select numerical features
```

```
X = data[['fare_amount', 'trip_distance']]
```

```
# Remove missing values
```

```
X = X.dropna()
```

```
# Train Isolation Forest
```

```
iso = IsolationForest(  
    contamination=0.02,  
    random_state=42  
)
```

```
# Predict anomalies
```

```
X['Anomaly'] = iso.fit_predict(X)
```

```
# Normal points and anomalies
```

```
normal = X[X['Anomaly'] == 1]
```

```
anomaly = X[X['Anomaly'] == -1]
```

```
# Plot the results
```

```
plt.figure(figsize=(8,6))
```

```
plt.scatter(normal['fare_amount'],
```

```
normal['trip_distance'],  
c='blue',  
s=20,  
label='Normal')
```

```
plt.scatter(anomaly['fare_amount'],  
            anomaly['trip_distance'],  
            c='red',  
            s=40,  
            label='Anomaly')
```

```
plt.title("Isolation Forest - NYC Taxi Fare Anomaly Detection")  
plt.xlabel("Fare Amount")  
plt.ylabel("Trip Distance")  
plt.legend()  
plt.show()
```

```
# Print number of anomalies
```

```
print("Total anomalies detected:", len(anomaly))
```

Output - Total anomalies detected: 312

Question 8: Train a SARIMA model on the monthly airline passengers dataset. Forecast the next 12 months and visualize the results. (Include your Python code and output in the code box below.)

Ans:- # Import required libraries

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from statsmodels.datasets import get_rdataset
```

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

```
# Load the AirPassengers dataset
```

```
data = get_rdataset("AirPassengers").data
```

```
# Create a datetime index
```

```
data["Month"] = pd.date_range(start="1949-01", periods=len(data),  
freq="MS")
```

```
data.set_index("Month", inplace=True)
```

```
# Time series
```

```
ts = data["value"]
```

```
# Train SARIMA model
```

```
model = SARIMAX(  
    ts,  
    order=(1, 1, 1),  
    seasonal_order=(1, 1, 1, 12)  
)
```

```
results = model.fit()
```

```
# Forecast next 12 months
```

```
forecast = results.forecast(steps=12)
```

```
# Create future dates
```

```
future_dates = pd.date_range(  
    start=ts.index[-1] + pd.DateOffset(months=1),  
    periods=12,  
    freq="MS"  
)
```

```
# Plot original series and forecast
```

```
plt.figure(figsize=(10,6))
```

```
plt.plot(ts, label="Original Data")
plt.plot(future_dates, forecast, color="red",
         label="12-Month Forecast")

plt.title("SARIMA Forecast for AirPassengers Dataset")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.legend()
plt.grid(True)
plt.show()
```

```
# Print forecast values
```

```
print(forecast)
```

```
1961-01-01    445.8
```

```
1961-02-01    420.6
```

```
1961-03-01    458.9
```

```
1961-04-01    495.7
```

```
1961-05-01    518.3
```

```
1961-06-01    593.2
```

```
1961-07-01    681.5
```

1961-08-01 670.4

1961-09-01 573.6

1961-10-01 522.1

1961-11-01 455.9

1961-12-01 503.8

dtype: float64

Question 9: Apply Local Outlier Factor (LOF) on any numerical dataset to detect anomalies and visualize them using matplotlib. (Include your Python code and output in the code box below.)

Ans:- # Import required libraries

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_wine
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neighbors import LocalOutlierFactor
```

```
# Load the Wine dataset
```

```
wine = load_wine()
```

```
X = wine.data
```

```
# Standardize the data
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
# Apply Local Outlier Factor (LOF)
```

```
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05)
```

```
# Predict anomalies
```

```
labels = lof.fit_predict(X_scaled)
```

```
# Separate normal points and anomalies
```

```
normal = X_scaled[labels == 1]
```

```
anomalies = X_scaled[labels == -1]
```

```
# Plot the results using the first two features
```

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(normal[:, 0], normal[:, 1],
```

```
            c='blue',
```

```
            s=40,
```

```
            label='Normal Data')
```

```
plt.scatter(anomalies[:, 0], anomalies[:, 1],  
            c='red',  
            marker='x',  
            s=80,  
            label='Anomalies')
```

```
plt.title("Local Outlier Factor (LOF) - Anomaly Detection")  
plt.xlabel("Feature 1 (Standardized)")  
plt.ylabel("Feature 2 (Standardized)")  
plt.legend()  
plt.grid(True)  
plt.show()
```

```
# Print the number of anomalies
```

```
print("Number of anomalies detected:", sum(labels == -1))
```